

Week 11 – Scheduling, Archiving and the Linux Boot Process

Week 11 covers:

- RH134 Chapter 2: Scheduling Future Tasks
 - RH134 Chapter 4: Archive and Transfer Files
 - RH134 Chapter 10: Control the Boot Process
 - Cron and Anacron
 - `tar` archives and compression
 - Linux boot stages
 - GRUB 2
 - Root-password recovery
-

1. Scheduling tasks with Cron

Cron is used to run repetitive tasks automatically at specified times.

Cron consists of:

Component	Purpose
<code>crond</code>	Background daemon that checks and runs scheduled jobs
Crontab	File containing scheduled jobs
<code>crontab</code> command	Used to view and edit a user's scheduled jobs
<code>/etc/crontab</code>	System-wide cron configuration
<code>/etc/cron.d</code>	Additional system cron files

Cron jobs should normally use non-interactive commands because no user may be present when they execute.

Check whether Cron is running:

```
systemctl status crond
```

Start and enable it:

```
sudo systemctl enable --now crond
```

2. Crontab format

A normal user crontab entry contains six parts:

Minute Hour Day-of-month Month Day-of-week Command

Example:

```
30 8 10 6 * /home/user/backup.sh
```

This runs the script at **8:30 AM on 10 June**.

Field	Valid values
Minute	0–59
Hour	0–23
Day of month	1–31
Month	1–12
Day of week	0–7, where 0 and 7 are Sunday

Command

Command or script to execute

The format shown in the slides is:

minute hour day-of-month month day-of-week command

3. Cron symbols

Symbol	Meaning
*	Every possible value
,	Several specific values
-	A range
/	An interval

Examples:

Run every day at 2:00 AM:

```
0 2 * * * /home/user/backup.sh
```

Run at 11:00 AM and 4:00 PM daily:

```
0 11,16 * * * /home/user/script.sh
```

Run every hour from 9:00 AM to 6:00 PM:

```
0 9-18 * * * /home/user/check.sh
```

Run every ten minutes:

```
*/10 * * * * /home/user/check-disk.sh
```

The examples in the slides demonstrate lists, ranges and step values.

4. User crontab commands

Edit the current user's crontab:

```
crontab -e
```

List scheduled jobs:

```
crontab -l
```

Remove the entire current user crontab:

```
crontab -r
```

Use this carefully because it removes every cron entry.

The root user can view another user's crontab:

```
sudo crontab -u username -l
```

User crontab files are stored under:

```
/var/spool/cron
```

They should be managed using **crontab**, not edited directly.

5. System crontab versus user crontab

A user crontab entry:

```
0 2 * * * /home/user/backup.sh
```

A system crontab entry includes an additional **username field**:

```
0 2 * * * root /home/user/backup.sh
```

System format:

```
minute hour day month weekday user command
```

The system-wide crontab is:

`/etc/crontab`

Additional system cron jobs may be stored in:

`/etc/cron.d`

Cron also commonly uses:

1. `/etc/cron.hourly`
2. `/etc/cron.daily`

`/etc/cron.weekly`

The slides state that `crond` checks the cron files every minute, and there is no need to restart it after changing crontab entries.

6. Cron variables

A crontab may contain variables such as:

3. `SHELL=/bin/bash`
4. `PATH=/usr/local/bin:/usr/bin:/bin`

`MAILTO=user@example.com`

`PATH` is important because Cron does not read the user's normal shell initialisation files.

For reliability, use full command paths:

`0 2 * * * /usr/bin/tar -cf /home/user/backup.tar /home/user/documents`

rather than relying on Cron to locate `tar`.

7. Cron keywords

Cron supports keywords that replace the five time fields.

Keyword	Meaning
---------	---------

@reboot	Run once during startup
@yearly	Run once per year
@annually	Same as @yearly
@monthly	Run once per month
@weekly	Run once per week
@daily	Run once per day
@midnight	Same as @daily
@hourly	Run once per hour

Example:

```
@reboot /home/user/startup.sh
```

8. Controlling access to Cron

The administrator can restrict Cron access using:

5. /etc/cron.allow

/etc/cron.deny

Rules from the slides:

- If only `cron.allow` exists, only listed users may use `crontab`.
 - If only `cron.deny` exists, listed users are denied.
 - If both exist, `cron.allow` takes priority.
 - Only root can modify these files.
-

9. Anacron

Anacron is designed for systems that are not continuously powered on.

Cron may miss a job if the system is switched off at the scheduled time. Anacron checks whether a periodic job was missed and runs it after the system starts.

The configuration file is:

`/etc/anacrontab`

General entry format:

period delay job-identifier command

Field	Meaning
Period	Number of days between executions
Delay	Minutes to wait after Anacron starts
Job identifier	Unique job name
Command	Command or script to run

Example:

```
3 10 cleanup-job /home/user/cleanup.sh
```

This means:

- Run once every three days.
- Wait ten minutes after Anacron starts.
- Use `cleanup-job` as the identifier.
- Execute `/home/user/cleanup.sh`.

Anacron records when jobs were last run under:

```
/var/spool/anacron
```

10. Creating archives with `tar`

`tar` combines several files and directories into one archive.

Create an archive:

```
tar -cf backup.tar documents/
```

Options:

Option	Meaning
<code>-c</code>	Create an archive
<code>-f</code>	Specify the archive filename
<code>-v</code>	Display processed files
<code>-t</code>	List archive contents

-x

Extract an archive

The -f option should be followed by the archive filename.

Example:

```
tar -cvf backup.tar /home/user/documents
```

11. Listing archive contents

View the contents without extracting:

```
tar -tf backup.tar
```

Detailed listing:

```
tar -tvf backup.tar
```

The slides state that -t lists the archive contents and -v provides more detail.

12. Extracting an archive

Extract into the current directory:

```
tar -xf backup.tar
```

Extract into another directory:

```
6. mkdir -p /home/user/recovered
```

```
tar -xf backup.tar -C /home/user/recovered
```

The -C option changes to the specified destination before extraction.

Always check archive contents before extracting an unfamiliar archive:

```
tar -tf backup.tar
```

13. Compressing archives

A `.tar` file is an archive but is not necessarily compressed.

Common compression tools:

Tool	Extension
<code>gzip</code>	<code>.gz</code>
<code>bzip2</code>	<code>.bz2</code>
<code>xz</code>	<code>.xz</code>

Compress with `gzip`:

```
gzip backup.tar
```

Final filename:

```
backup.tar.gz
```

The original `backup.tar` is normally replaced by `backup.tar.gz`.

Decompress:

```
gunzip backup.tar.gz
```

14. Creating compressed archives directly

Create a `gzip`-compressed tar archive:

```
tar -czf backup.tar.gz documents/
```

Create a `bzip2`-compressed archive:

```
tar -cjf backup.tar.bz2 documents/
```

Create an xz-compressed archive:

```
tar -cJf backup.tar.xz documents/
```

Option	Compression
-z	gzip
-j	bzip2
-J	xz

Extract gzip:

```
tar -xzf backup.tar.gz
```

Extract bzip2:

```
tar -xjf backup.tar.bz2
```

Extract xz:

```
tar -xJf backup.tar.xz
```

The final-slide lab specifically uses separate `tar` and `gzip` commands.

15. Transferring files securely

The chapter title includes transferring files, but the final-slide questions only test archiving and extraction.

A common secure transfer command is:

```
scp file user@remote-host:/destination/
```

Copy a directory recursively:

```
scp -r directory user@remote-host:/destination/
```

This is supplementary background; it is not required by the final-slide exercises shown in the uploaded summary.

16. Linux boot process

The slides divide the Linux boot process into four main stages:

1. Firmware
2. Bootloader
3. Kernel
4. Init

Firmware stage

The system performs hardware checks and identifies a bootable device using BIOS or UEFI.

Bootloader stage

GRUB loads and presents available boot entries.

Kernel stage

The selected Linux kernel and `initramfs` are loaded.

Init stage

The first user-space process starts and brings up services, terminals and the login screen.

On modern Red Hat systems, the init system is normally `systemd`.

17. Checking the system boot time

Use:

```
who -b
```

Example:

```
system boot 2026-07-23 09:14
```

Alternative command:

```
uptime -s
```

The final-slide question specifically asks for **who**, so the expected answer is:

who -b

18. Viewing boot logs

View boot sessions recorded by the system journal:

```
journalctl --list-boots
```

View the current boot log:

```
journalctl -b
```

View the previous boot:

```
journalctl -b -1
```

List only the final five records from the boot-session list:

```
journalctl --list-boots | tail -n 5
```

This is the closest direct answer to the final-slide instruction to list the last five boot logs.

19. GRUB 2 files

Important GRUB 2 locations from the slides:

Location	Purpose
<code>/boot/grub2/grub.cfg</code>	Generated main GRUB configuration
<code>/etc/default/grub</code>	Administrator customisation settings

<code>/etc/grub.d</code>	Scripts used to generate configuration
<code>/boot/grub2</code>	GRUB modules and configuration files
<code>/sbin/grub2-mkconfig</code>	Regenerates <code>grub.cfg</code>

The slides warn that `/boot/grub2/grub.cfg` should not normally be edited manually.

View it:

```
sudo less /boot/grub2/grub.cfg
```

View administrator settings:

```
cat /etc/default/grub
```

Regenerate the configuration:

```
sudo grub2-mkconfig -o /boot/grub2/grub.cfg
```

Only perform regeneration when configuration changes are intended.

7.